



SOFTWARE DEVELOPMENT: Technical documentation

PREPARED FOR

PREPARED BY

FOSSO KENGNI Doriane

MARCHÉHUB – Technical Specifications (Final Version)

Phase 1: Design & Planning (Revisited)

1. Project Objective

The goal is to develop an MVP of **MARCHÉHUB**, a responsive web platform that connects producers and buyers across Sub-Saharan Africa.

The application must:

- Utilize at least **10 KendoReact components**.
 - Integrate **Nuclia API** for intelligent semantic search.
 - Comply with accessibility and performance best practices.
-

2. Functional Requirements

FR1: User Management

- **FR1.1:** The system shall allow users to register and log in using email and password.
- **FR1.2:** The system shall allow users to create and edit their profile (account type, name, location, bio, contact details, and accepted payment methods).
- **FR1.3:** The system shall allow users to log out.

FR2: Catalog Management

- **FR2.1:** The system shall allow sellers to publish, edit, and delete products (title, description, images, price, category, quantity).
- **FR2.2:** The system shall allow all users to browse a paginated product list.
- **FR2.3:** The system shall provide filtering by category and location.

FR3: Advanced Search

- **FR3.1:** The system shall integrate the **Nuclia API** to support natural language semantic search across products and profiles.

FR4: Communication System

- **FR4.1:** The system shall allow authenticated users to send private messages to sellers (linked or not to a specific product).
- **FR4.2:** The system shall allow users to view their inbox and sent conversations.

FR5: Review System

- **FR5.1:** The system shall allow buyers to leave a review (rating 1–5 and comment) on a seller's profile.
- **FR5.2:** The system shall display average ratings and reviews on seller profiles.

3. Non-Functional Requirements

- **NF1: Accessibility** – The application shall comply with **WCAG 2.1 AA** standards.
- **NF2: Performance** – Page load times for main views shall be under **3 seconds** on a 3G connection.

- **NF3: Usability** – The UI shall be intuitive and accessible for a non-tech-savvy audience.
 - **NF4: Compatibility** – The application shall be responsive and work across modern desktop and mobile browsers.
 - **NF5: Maintainability** – Code shall be well-structured, commented, and strongly typed using TypeScript.
 - **NF6: Security** – Passwords shall be hashed with **bcrypt**; API routes shall be secured with **JWT authentication**.
 - **NF7: Offline Tolerance (Future Improvement)** – The system shall cache previously loaded data for offline access.
-

4. Finalized Tech Stack

- **Frontend:** React 18, TypeScript, Vite, KendoReact, TanStack Query, React Hook Form, Zod
 - **Backend:** Node.js, Express, Prisma, PostgreSQL, JWT
 - **Search:** Nuclia API
 - **File Storage:** Cloudinary
 - **Deployment:** Netlify (Frontend), Railway (Backend & Database)
 - **Code Management:** GitHub Monorepo
-

Phase 2: Analysis & Modeling

1. Use Case Diagram (Summary)

Actors: Anonymous User, Authenticated User, Seller, Buyer

Main Use Cases:

- Register, Log in, View Catalog, Search Products (Basic User)
 - Manage Profile, Manage Products (Seller)
 - Contact Seller, Leave Review (Buyer)
-

2. Class/Entity Diagram (Data Model)

erDiagram

```
USER {  
    string id PK  
    string email  
    string passwordHash  
    string name  
    string avatarUrl  
    string location  
    string bio  
    string phone  
    string paymentMethods  
    string role  
    datetime createdAt  
}  
  
PRODUCT {  
    string id PK  
    string title
```

```
    string description

    float price

    string[] images

    int quantity

    string categoryId FK

    string userId FK

    datetime createdAt
}

CATEGORY {
    string id PK

    string name
}

MESSAGE {
    string id PK

    string content

    string senderId FK

    string receiverId FK

    string productId FK NULL

    datetime createdAt
}

REVIEW {
    string id PK

    int rating

    string comment
```

```
string authorId FK
string sellerId FK
datetime createdAt
}
```

```
USER ||--o{ PRODUCT : "sells"
USER ||--o{ MESSAGE : "sends"
USER ||--o{ MESSAGE : "receives"
USER ||--o{ REVIEW : "writes"
USER ||--o{ REVIEW : "receives"
CATEGORY ||--o{ PRODUCT : "contains"
PRODUCT }o--|| MESSAGE : "referenced in"
```

Relationships Description:

- A **User** can publish multiple **Products**.
 - A **User** can send and receive multiple **Messages**.
 - A **User (Buyer)** can write multiple **Reviews**.
 - A **User (Seller)** can receive multiple **Reviews**.
 - A **Category** contains multiple **Products**.
 - A **Message** may optionally reference a **Product**.
-

Phase 3: Detailed Design & Architecture

3.1. Database Schema (Prisma Schema)

The following schema is the direct implementation of our conceptual data model using **Prisma ORM**.

The schema file is located at: `server/prisma/schema.prisma`.

```
// server/prisma/schema.prisma

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id          String    @id @default(cuid())
  email       String    @unique
  passwordHash String
  name        String
```



```

        avatarUrl      String?
        @default("https://res.cloudinary.com/.../default_avatar.png")

        location       String

        bio            String?

        phone          String?

        paymentMethods String?

        role           String    @default("USER")

// Relations

products             Product[]  @relation("UserProducts")
sentMessages         Message[]  @relation("SentMessages")
receivedMessages     Message[]  @relation("ReceivedMessages")
writtenReviews       Review[]   @relation("UserReviews")
receivedReviews      Review[]   @relation("SellerReviews")

createdAt DateTime @default(now())

@@map("users")
}

```

```

model Product {
    id          String    @id @default(cuid())
    title       String
    description  String
    price       Float
}

```

```

    images      String[]
    quantity    Int      @default(1)
    categoryId   String
    userId       String

    category Category @relation(fields: [categoryId], references:
[id], onDelete: Cascade)

    user      User      @relation("UserProducts", fields: [userId],
references: [id], onDelete: Cascade)
    messages Message[]

    createdAt DateTime @default(now())

    @@map("products")
}

model Category {
    id          String    @id @default(cuid())
    name        String    @unique
    products    Product[]

    @@map("categories")
}

model Message {

```

```

    id          String    @id @default(cuid())
    content     String
    senderId    String
    receiverId  String
    productId   String?

    sender User    @relation("SentMessages", fields: [senderId],
references: [id], onDelete: Cascade)

    receiver User  @relation("ReceivedMessages", fields:
[receiverId], references: [id], onDelete: Cascade)

    product Product? @relation(fields: [productId], references:
[id], onDelete: Cascade)

    createdAt DateTime @default(now())

    @@map("messages")
}

```

```

model Review {
    id          String    @id @default(cuid())
    rating      Int
    comment     String?
    authorId    String
    sellerId    String
}

```

```

    author User @relation("UserReviews", fields: [authorId],
references: [id], onDelete: Cascade)

    seller User @relation("SellerReviews", fields: [sellerId],
references: [id], onDelete: Cascade)

    createdAt DateTime @default(now())

    @@unique([authorId, sellerId])

    @@map("reviews")
}

```

3.2. API Architecture (RESTful Endpoints)

Method	Endpoint	Description	Auth Required
Auth			
POST	/api/auth/register	Register a new user	No
POST	/api/auth/login	Authenticate user	No
GET	/api/auth/me	Get the profile of the logged-in user	Yes
Users			

GET	/api/users	Retrieve a list of sellers (discovery)	No
GET	/api/users/:id	Get a seller's public profile	No
PUT	/api/users/profile	Update the authenticated user's profile	Yes
Products			
GET	/api/products	Retrieve a list of products (with filters)	No
GET	/api/products/:id	Retrieve product details	No
POST	/api/products	Create a new product	Yes
PUT	/api/products/:id	Update a product	Yes (Owner only)
DELETE	/api/products/:id	Delete a product	Yes (Owner only)
Messages			
POST	/api/messages	Send a message to a seller	Yes
GET	/api/messages/inbox	Retrieve inbox of the authenticated user	Yes
Reviews			

POST	/api/reviews	Post a review on a seller	Yes
GET	/api/reviews/seller/:sellerId	Retrieve reviews for a seller	No
Search			
GET	/api/search?q=...	Perform semantic search (Nuclia API)	No
Categories			
GET	/api/categories	Retrieve all categories	No

The backend API is built with **Express.js** and follows RESTful conventions. Below is the list of endpoints grouped by domain.

3.3. Wireframes (Low-Fidelity Mockups)

Home Page / Catalog

```

-----
| [Logo] [Search] [Login]      |
-----
| [Filters: Categories, Location]|
-----
| [Product Card 1] [Product 2] |
| [Image][Title][Price][Location]|
|                               |

```

[Product Card 3] [Product 4]	
[Pagination]	

Product Detail Page

[Logo] [Search] [Profile]	
---------------------------	--

[< Back]	
----------	--

[Image Gallery]	
-----------------	--

--	--

Product Title	
---------------	--

Price	
-------	--

Long Description	
------------------	--

--	--

[Seller: Avatar + Name]	
-------------------------	--

[Rating: ★★★★★☆]	
------------------	--

[Payment Methods]	
-------------------	--

--	--

[Contact Button]	
------------------	--

Contact Popup (Kendo Dialog)

```
-----  
  
| Contact [Seller Name]          |  
| About [Product Title]         |  
-----  
  
| [Your message...] (Textarea)  |  
-----  
  
| [Send Button] [Cancel]        |  
-----
```

Seller Profile Page

```
-----  
  
| [Banner]                      |  
| [Avatar] [Name]               |  
| [Average Rating] [# Reviews] |  
| [Location]                    |  
| [Bio]                         |  
| [Payment/Contact Info]        |  
-----  
  
| [Product List]                |  
|                               |  
-----  
  
| [Reviews Section]             |  
| [Review 1] [Review 2] ...    |  
-----
```


3.4. Component & Deployment Architecture

3.4.1. Component Architecture

The system follows a **modular architecture** based on **React (frontend)**, **Express.js (backend API)**, and **Prisma + PostgreSQL (database)**.

Each component is designed to be **loosely coupled** and **highly cohesive**.

flowchart TB

```
subgraph Client["Frontend (React + KendoReact)"]
    UI[UI Components (KendoReact, Forms, Product Cards)]
    Auth[Authentication Module]
    Catalog[Catalog & Filtering Module]
    Search[Semantic Search (Nuclia API Integration)]
    Messaging[Messaging UI]
    Reviews[Reviews & Ratings UI]
    Profile[User Profile Management]
end
```

end

```
subgraph API["Backend API (Node.js + Express)"]
    AuthAPI[/Auth Controller/]
    UserAPI[/User Controller/]
    ProductAPI[/Product Controller/]
    MessageAPI[/Message Controller/]
    ReviewAPI[/Review Controller/]
    SearchAPI[/Search Controller (Nuclia)/]
    CategoryAPI[/Category Controller/]
end
```

end

subgraph DB["Database (PostgreSQL + Prisma)"]

UsersTable[(Users)]

ProductsTable[(Products)]

CategoriesTable[(Categories)]

MessagesTable[(Messages)]

ReviewsTable[(Reviews)]

end

UI --> Auth

UI --> Catalog

UI --> Search

UI --> Messaging

UI --> Reviews

UI --> Profile

Auth --> AuthAPI

Profile --> UserAPI

Catalog --> ProductAPI

Messaging --> MessageAPI

Reviews --> ReviewAPI

Search --> SearchAPI

```
AuthAPI --> UsersTable
UserAPI --> UsersTable
ProductAPI --> ProductsTable
ProductAPI --> CategoriesTable
MessageAPI --> MessagesTable
ReviewAPI --> ReviewsTable
SearchAPI --> Nuclia["Nuclia API"]

DB <--> Prisma["Prisma ORM"]
```

Key Points:

- **Frontend:** Built with React 18, TypeScript, and KendoReact components. Handles UI, form validation, and client-side caching (TanStack Query).
- **Backend:** Organized into domain-based controllers (Auth, Users, Products, Messages, Reviews, Search, Categories). Implements RESTful APIs.
- **Database:** PostgreSQL managed via Prisma ORM. Schema designed for relational integrity and scalability.
- **Search:** Delegated to **Nuclia API** for semantic and natural language search.
- **Cloudinary:** Used for media storage (images, avatars).

3.4.2. Deployment Architecture

The deployment environment leverages **cloud-based services** with CI/CD via GitHub.

flowchart LR

```
subgraph ClientSide["Client Devices"]
    Browser["Web Browser (Mobile/Desktop)"]
end
```

```
subgraph Hosting["Hosting & Infrastructure"]
    Netlify["Netlify (Frontend Hosting)"]
    Railway["Railway (Backend + DB)"]
    Cloudinary["Cloudinary (Media Storage)"]
    NucliaAPI["Nuclia API (Semantic Search SaaS)"]
end
```

```
subgraph Repo["Development Workflow"]
    GitHub["GitHub Monorepo"]
    CI["CI/CD Pipeline"]
end
```

```
Browser --> Netlify
Netlify --> Railway
Netlify --> Cloudinary
Railway --> PostgreSQL["PostgreSQL Database"]
Railway --> Cloudinary
Railway --> NucliaAPI
GitHub --> CI --> Netlify
GitHub --> CI --> Railway
```

Deployment Flow:

1. **Frontend** is deployed on **Netlify** for fast, global CDN delivery.
2. **Backend + Database** are hosted on **Railway**, providing scalability and managed services.
3. **Media Files (images, avatars, product pictures)** are stored and delivered via **Cloudinary**.
4. **Semantic Search** is powered by **Nuclia API** (external service).
5. **GitHub Monorepo** ensures all code (frontend + backend) is centralized. CI/CD pipelines automatically deploy to Netlify and Railway on merge.

